模型压缩 前置知识

> 这份文档为零基础读者准备。如果你已经熟悉量化、蒸馏、剪枝等概念，可以直接翻阅主报告。

> 🎯 **读完这篇你能**：在自己的消费级硬件（如单张 24GB 显存显卡）上，选择一个 70B 级开源模型的合适量化格式（如 Q4_K_M），完成部署并在质量损失可接受的范围内跑通推理。

一、模型压缩到底是什么？用一句话说清楚

模型压缩 = 把大模型变小，让它能在你的电脑上跑起来。

一个 70B 参数的 Llama 3 模型，用 FP16（半精度浮点数）存储需要约 140GB 显存——你需要 5–6 块 A100 80G 显卡才能跑起来。绝大多数个人开发者和中小企业不可能有这种硬件。

模型压缩的目标就是：在尽量保持模型质量的前提下，大幅减小模型的大小和计算需求。

三个主要方法：

方法	一句话	类比
量化（Quantization）	降低数字精度	一张照片从 4K 缩到 1080p，还是能认出来是什么
蒸馏（Distillation）	让好老师教出好学生	学霸的笔记给同学看，同学学得比直接看书快
剪枝（Pruning）	去掉不重要的连接	一棵树剪掉枯枝，养分集中在有用的枝条上

二、量化（Quantization）

精度是什么？

模型参数默认用 FP16（16-bit 浮点数）存储——每个参数占 2 个字节。70B 模型 = $70 \times 10^9 \times 2$ bytes = 140GB。

4-bit 量化后：每个参数占 0.5 个字节。70B 模型 = $70 \times 10^9 \times 0.5$ bytes = 35GB——降到原来的 **1/4**。

这就是量化的核心逻辑：**用更少的比特数表示模型参数，显存需求直线下降，推理速度显著提升。**

常见的量化格式

格式	位宽	70B 模型大小	质量
FP16（原始）	16-bit	140 GB	基准
INT8 (Q8_0)	8-bit	70 GB（社区估算）	几乎无损
INT4 (Q4_K_M)	4-bit	40 GB（社区估算）	极小损失（社区推荐）
INT2 (Q2_K)	2-bit	25 GB（社区估算）	质量下降明显

Q4_K_M 是 2026 年社区公认的最佳平衡点——质量几乎看不出差别，显存需求降低 70%+（社区估算）。

两种量化方式

训练后量化（PTQ, Post-Training Quantization）：模型训练完后直接量化。最常用。GPTQ、AWQ、GGUF 都是 PTQ。

量化感知训练（QAT, Quantization-Aware Training）：在训练过程中就模拟量化效果，让模型"学会"适应低精度。效果更好但需要重新训练或微调。

三、主流量化方法

困惑度（Perplexity）

量化方法优劣用什么衡量？主报告中反复出现一个指标——**困惑度（Perplexity）**。它衡量模型对下一个词的"不确定程度"：困惑度越低，模型越确定，质量越高。量化前后的困惑度变化，就是量化质量的标尺。一个优秀的量化方法，困惑度上升应不超过 0.5（社区估算）。

GPTQ（GPT-Post-training Quantization，GPT 后训练量化，2022）

基于二阶优化的方法。先用一小批校准数据（128 个样本就够了，Frantar et al., 2023）分析模型参数，然后逐层将 FP16 转换为 INT4。Hugging Face 上最多的量化模型就是 GPTQ 格式。

类比：像一个老裁缝量体裁衣——先拿 128 个"样本人"量尺寸（校准数据），然后根据这些数据调整每件衣服的剪裁方案（逐层量化），而不是所有人统一剪裁。

AWQ（Activation-aware Weight Quantization，激活感知权重量化，2023）

观察激活值的分布来识别"重要"的权重通道——只保护重要的 1% 通道不量化，其余大胆量化。AWQ 的推理速度通常比 GPTQ 快 10-20%（Lin et al., 2023）。

SmoothQuant（Xiao et al., 2022）

LLM 的激活值中偶尔出现极端异常值（比正常值大 100 倍以上，Xiao et al., 2022），导致直接量化会把精度搞崩溃。SmoothQuant 把这些异常值"平滑"转移到权重上——激活值和权重量化都不再困难。它目前是 FP8/INT8 推理加速的

标配方案。

GGUF（GPT-Generated Unified Format，GPT 通用量化格式，2023）

llama.cpp 的原生格式。不需要校准数据，直接按块量化。配合 llama.cpp 可以在 CPU 或普通显卡上跑大模型。

类比：GGUF 就像一个"一键打包"方案——不需要知道冰箱里面有什么，只要知道冰箱总容积（总位宽），按比例压缩就行。简单粗暴但好用。

NF4（bitsandbytes）

Normal Float 4-bit——专门为神经网络权重分布设计的 4-bit 格式。QLoRA（让模型微调时显存大幅降低的技术）的核心就是 NF4。

FP8 量化（2024-2025）

H100/H200 等新 GPU 原生支持 FP8 计算（NVIDIA, 2022）——不需要额外转换，硬件直接算。FP8 推理速度约为 FP16 的 2 倍（社区估算），精度损失极小。2025-2026 年 FP8 已取代 BF16 成为行业标准的推理精度。

```
实用指南：
  服务端高吞吐（vLLM + GPU）→ AWQ
  Hugging Face 上模型最多 → GPTQ
  本地电脑/Mac 运行 → GGUF（Q4_K_M）
  Apple Silicon（Mac）→ GGUF 或 MLX
```

四、蒸馏（Distillation）

核心思想

训练一个小模型（学生）去模仿大模型（老师）的输出。学生不需要从零开始学习，而是直接从老师的行为中"抄作业"。

传统蒸馏（Hinton et al., 2015）：学生模仿老师输出的概率分布（logits），而不只是最终的答案。

LLM 时代的蒸馏：跟传统蒸馏有本质不同。不是让学生模仿"答案"或"概率分布"，而是让学生在老师生成的推理链（思考过程）上微调。DeepSeek-R1（2025）把 671B 模型的推理能力蒸馏到 7B 小模型上，在数学竞赛上接近 GPT-4 水平（社区估算）。

Phi 系列（Microsoft, 2023-2025） 走了另一条路：不是从单个老师蒸馏，而是用大模型生成海量"教科书质量"的合成数据来训练小模型。Phi-1（1.3B）用这种数据训练，性能就达到参数量 10x 模型的水平（社区估算）。这本质上是数据层面的蒸馏。

蒸馏 vs 量化

蒸馏和量化解决的是不同层面的问题，通常组合使用：

	蒸馏	量化
做什么	把知识从大模型转移到小模型	降低大模型的数值精度
效果	模型变小，但"更聪明"	模型大小减半/减 3/4，但智能水平不变
需要啥	需要大模型的训练数据/输出	只需模型本身，即量即用
组合	先蒸馏（70B→7B），再量化（7B→4-bit）	—

经典组合：DeepSeek-R1 蒸馏出 7B 模型 → 量化为 4-bit → 在普通 MacBook 上就能跑，数学推理能力还很强。

五、剪枝（Pruning）

核心思想

去掉模型中不重要的参数或结构。思想来自人脑——人脑中的突触会不断修剪，不常用的连接会被淘汰。

非结构化剪枝：把权重矩阵中接近 0 的单个值设为 0。稀疏存储，需要特殊硬件支持才能加速。

结构化剪枝：直接移除整个注意力头、层或神经元。不依赖特殊硬件，因为模型结构变小了。

在 LLM 上，剪枝的效果不如量化好——因为大模型看起来"处处都有用"，很难找到真正可以安全去掉的部分。SparseGPT（Frantar & Alistarh, 2023）和 Wanda（2023）是主要的剪枝方法。

六、一张图总结

```
          模型压缩三把刀
                |
    ┌───────────┼───────────┐
    │           │           │
   量化       蒸馏        剪枝
    │           │           │
  降精度     降参数量    降结构
    │           │           │
  ┌─┴─┐     ┌─┴─┐     ┌─┴─┐
  │   │     │   │     │   │
GPTQ AWQ 老师→学生 SparseGPT Wanda
GGUF NF4 Phi系列 R1蒸馏
    │           │           │
  ──────────────┼──────────────
                │
        实用组合策略
    • 端侧推理 → 量化 (Q4)
    • 特定领域 → 蒸馏 + 量化
    • Mac 本地 → GGUF
    • 生产服务 → AWQ (vLLM)
```

核心一句话：**2026 年，4-bit 量化是部署标配，蒸馏是领域特化利器，剪枝还在路上。**

参考来源

- **Distilling the Knowledge in a Neural Network** (Hinton et al., Google, 2015) — 知识蒸馏的开山之作，提出用教师模型的软标签训练学生模型 — https://arxiv.org/abs/1503.02531
- **LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale** (Dettmers et al., UW, 2022) — 8-bit 量化的里程碑，提出混合精度分解方案解决 LLM 异常值问题 — https://arxiv.org/abs/2208.07339
- **GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers** (Frantar et al., IST Austria, 2023) — 基于二阶优化的逐层量化方法，HuggingFace 上最广泛的量化格式之一 — https://arxiv.org/abs/2210.17323
- **AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration** (Lin et al., CMU, 2023) — 通过激活值分布识别并保护重要权重通道，推理速度比 GPTQ 快 10-20% — https://arxiv.org/abs/2306.00978
- **SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models** (Xiao et al., MIT/NVIDIA, 2022) — 将激活值异常值平滑转移到权重上，FP8/INT8 推理加速的标配方案 — https://arxiv.org/abs/2211.10438
- **SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot** (Frantar & Alistarh, IST Austria, 2023) — 一次性剪枝方法，无需重新训练即可稀疏化 50% 以上参数且质量几乎不降 — https://arxiv.org/abs/2301.00774

- **NVIDIA H100 Tensor Core GPU Architecture Whitepaper** (NVIDIA, 2022) — Hopper 架构原生支持 FP8 计算，FP8 推理速度约为 FP16 的 2 倍 — https://resources.nvidia.com/en-us-tensor-core/gtc22-whitepaper-hopper